

Modelos Generativos Profundos

Clase 7: Modelos autorregresivos y LLMs (parte III)

Fernando Fêtis Riquelme

Otoño, 2026

Facultad de Ciencias Físicas y Matemáticas
Universidad de Chile

Tokenizadores

Positional encoding

Mecanismos de normalización

Mecanismos de atención

Dropout y conexiones residuales

Introducción

En la clase anterior se estudió la arquitectura GPT, la cual permite entender el esqueleto general que siguen los modelos autorregresivos actuales.

Si bien este esqueleto general se ha mantenido a lo largo del tiempo, se han introducido muchas modificaciones a la arquitectura original, las cuales han permitido mejorar la eficiencia y rendimiento de los modelos.

Más aún, los LLMs solo son una aplicación de la arquitectura Transformer, la cual hoy en día se utiliza de manera transversal en el machine learning moderno.

En esta clase se revisarán algunas variaciones usuales de los componentes que constituyen la arquitectura Transformer.

Tokenizadores

Problema de tokenización

Dado un dominio de texto, es necesario elegir un **algoritmo de tokenización**, el cual definirá el vocabulario $\mathcal{V}$ sobre el que se tokenizará dicho corpus.

- Aumentar $|\mathcal{V}|$ aumenta la cantidad de parámetros del modelo.
- Disminuir $|\mathcal{V}|$ provoca que las secuencias de tokens sean más largas, lo cual aumenta el costo de procesamiento.

Encontrar la *mejor* tokenización es un problema NP-completo, por lo que se debe recurrir a métodos aproximados.

Byte pair encoding

Uno de los algoritmos de tokenización más usuales es el algoritmo **byte pair encoding (BPE)**:

- Comienza considerando cada caracter como un token de $\mathcal{V}$.
- Luego, forma nuevos tokens combinando cadenas de caracteres de acuerdo a su frecuencia de ocurrencia en el corpus.
- Para esto, calcula la frecuencia de todos los bigramas en el corpus y agrega al vocabulario el bigrama con mayor frecuencia.
- Este proceso se repite hasta que se alcanza el tamaño de vocabulario deseado.
- Otros algoritmos como **WordPiece** siguen enfoques similares.

Tokens especiales

Es usual incluir **tokens especiales** que cumplen funciones específicas en el modelo. Algunos ejemplos son:

- **<SEP>**: usado para separar dos partes de la secuencia.
- **<CLS>**: usado como embedding de la secuencia completa.
- **<MASK>**: usado para enmascarar partes de la secuencia.
- **<TOOL>**: usado para demarcar el uso de herramientas externas.
- **<SYSTEM>, <USER>, <ASSISTANT>**: usado para indicar roles dentro de una conversación.
- **<THINKING>**: usado para demarcar el proceso de razonamiento interno del modelo.

Positional encoding

Omitiendo el masking causal aplicado en el mecanismo de self-attention, un bloque Transformer es **equivariante bajo permutaciones**. Es decir, para una entrada $(x_1, \dots, x_T)$:

$$\text{Transformer}(\sigma(x_1, \dots, x_T)) = \sigma(\text{Transformer}(x_1, \dots, x_T)),$$

para toda permutación $\sigma : \{1, \dots, T\} \rightarrow \{1, \dots, T\}$.

En consecuencia, es necesario inyectar información posicional a la arquitectura para que esta pueda conocer el orden de los tokens dentro de la secuencia y no actúe simplemente como una **bag of embeddings**.

Embeddings posicionales absolutos

Una forma sencilla de inyectar información posicional es utilizando una capa de **embedding posicional absoluto** (APE), la cual asigna a cada posición $t \in \{1, \dots, T\}$ un vector de embedding $p_t \in \mathbb{R}^D$.

De esta forma, en lugar de alimentar el bloque Transformer con la secuencia de embeddings $(x_1, \dots, x_T)$, se alimenta con la secuencia $(x_1 + p_1, \dots, x_T + p_T)$.

Dado que una capa de embedding es una lookup table entrenable, esta técnica restringe la longitud máxima de las secuencias a un valor predefinido, $T_{\max} \in \mathbb{N}$, conocido como **ventana de contexto**.

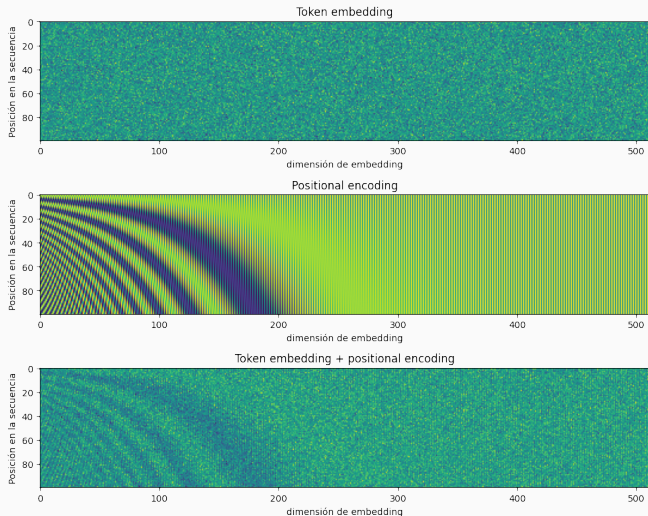
Embeddings posicionales sinusoidales

Otra opción para APEs es utilizar **embeddings sinusoidales**:

$$p_t = \begin{pmatrix} \sin(\omega_1 t) \\ \cos(\omega_1 t) \\ \vdots \\ \sin(\omega_{D/2} t) \\ \cos(\omega_{D/2} t) \end{pmatrix} \in [-1, 1]^D$$

- Las **frecuencias de oscilación**, $\omega_1, \dots, \omega_{D/2} > 0$, suelen elegirse siguiendo una progresión geométrica decreciente.
- Es decir, $\omega_k = b^{-\frac{k}{D}}$, donde $b > 1$ es un hiperparámetro.
- Notar que este mecanismo no utiliza parámetros entrenables.

Embeddings posicionales sinusoidales



Embeddings posicionales relativos

Una posible variación de los APEs es utilizar **embeddings posicionales relativos** (RPE), los cuales, en vez de asignar un embedding a cada posición, asignan un embedding a cada posible distancia entre posiciones.

Para esto, notar que el único lugar donde interactúan los embeddings de distintas posiciones dentro de un bloque Transformer es el mecanismo de atención.

Si bien se han propuesto muchas formas de implementar RPEs, se revisarán dos de las más usuales:

- **Attention with linear biases (ALiBi).**
- **Rotary positional embeddings (RoPE).**

Attention with linear biases

En el mecanismo de atención usual, se calculan los scores de atención como $s_{ij} = \frac{\langle q_i, k_j \rangle}{\sqrt{P}}$.

ALiBi modifica esta fórmula para incluir información posicional relativa:

$$s_{ij} = \frac{\langle q_i, k_j \rangle}{\sqrt{P}} + m|i - j|,$$

donde $m > 0$ (**pendiente de sesgo**) es un hiperparámetro definido para cada cabezal de atención, provocando un trade-off entre atención global y local.

Rotary positional embeddings

RoPE es un mecanismo de RPE que busca que el producto de atención solo dependa de los embedding x_i y x_j , y de la distancia $i - j$.

Es decir, busca funciones $f_q, f_k : \mathbb{R}^D \times \mathbb{N} \rightarrow \mathbb{R}^D$ para definir los vectores query y key como

$$\begin{cases} q_i = f_q(x_i, i) \\ k_j = f_k(x_j, j) \end{cases},$$

de modo que el producto de atención se pueda escribir como

$$\langle q_i, k_j \rangle = g(x_i, x_j, i - j),$$

para alguna función $g : \mathbb{R}^D \times \mathbb{R}^D \times \mathbb{Z} \rightarrow \mathbb{R}$.

Rotary positional embeddings

Una solución para este problema es definir f_q y f_k como rotaciones de los embeddings x_i y x_j respectivamente, donde el ángulo de rotación depende de la posición del token:

$$f_{\{q,k\}}(x_k, k) = R_{\Theta}(k)W_{\{q,k\}}x_k,$$

donde $R_{\Theta}(k) \in \mathcal{M}_{D,D}(\mathbb{R})$ es una matriz de rotaciones.

De este modo, el producto de atención se puede escribir como

$$\langle q_i, k_j \rangle = (W_q x_i)^{\top} R_{\Theta}(i - j) W_k x_j,$$

lo que cumple la condición deseada.

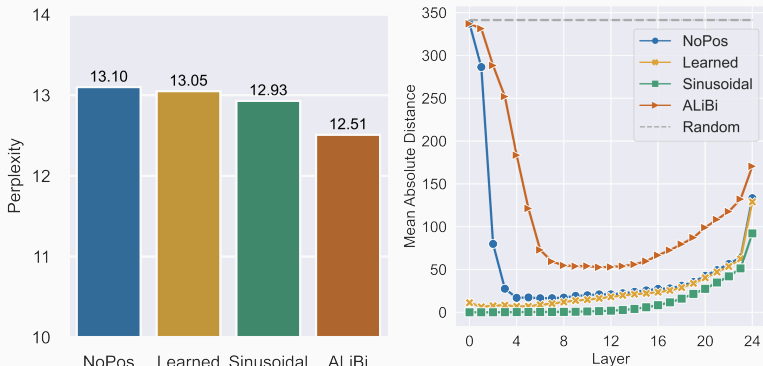
Rotary positional embeddings

Para un conjunto de frecuencias $\Theta = \{\theta_1, \dots, \theta_{D/2}\}$, la matriz de rotación $R_\Theta(k) \in \mathcal{M}_{D,D}(\mathbb{R})$ de RoPE es una matriz bloque-diagonal, donde cada bloque es una matriz de rotación 2D con ángulo $k\theta_d$:

$$R_\Theta(k) = \begin{pmatrix} \cos(k\theta_1) & -\sin(k\theta_1) & \cdots & 0 & 0 \\ \sin(k\theta_1) & \cos(k\theta_1) & \cdots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & \cos(k\theta_{D/2}) & -\sin(k\theta_{D/2}) \\ 0 & 0 & \cdots & \sin(k\theta_{D/2}) & \cos(k\theta_{D/2}) \end{pmatrix}$$

No positional encoding (NoPE)

Por otro lado, si bien un bloque Transformer es equivarente bajo permutaciones, resulta ser que el mecanismo de masking causal sí inyecta información posicional implícita:



Mecanismos de normalización

Capas de normalización

Una de las principales innovaciones que permitió entrenar CNNs profundas fue el uso de **batch normalization**.

En el caso de modelos tipo GPT, los cuales también están formados por varias capas neuronales, también es necesario agregar capas de normalización.

En este caso, es usual normalizar cada token de manera individual (i.e., sin considerar otros tokens u otras secuencias). Las dos alternativas más frecuentes son:

- **Layer normalization (LN).**
- **Root mean square layer normalization (RMSNorm).**

Layer normalization

Layer normalization normaliza (en media y varianza) cada vector de embedding individualmente:

$$\text{LayerNorm}(x) := \alpha \odot \tilde{x} + \beta, \quad \text{donde} \quad \tilde{x} = \frac{x - \text{mean}(x)}{\text{std}(x)}$$

La ubicación de esta normalización dentro de los bloques MHSA y FF influye en la estabilidad del entrenamiento:

- **Post-LN:** es de la forma $x \mapsto \text{LayerNorm}(x + \text{SubLayer}(x))$
- **Pre-LN:** es de la forma $x \mapsto x + \text{SubLayer}(\text{LayerNorm}(x))$

Hoy en día se prefiere la arquitectura Pre-LN ya que genera una dinámica de entrenamiento más estable.

Mecanismos de normalización

Otra técnica de normalización usual es **RMS normalization**, la cual calcula, para cada vector de embedding $x \in \mathbb{R}^D$:

$$\text{RMSNorm}(x) = \alpha \odot \frac{x}{\text{rms}(x)}, \quad \text{donde} \quad \text{rms}(x) = \sqrt{\frac{1}{D} \sum_{i=1}^D x_i^2}$$

- Notar que $\alpha \in \mathbb{R}^D$ es el único parámetro entrenable.
- RMSNorm normaliza todos los embeddings a una misma esfera de radio $\frac{1}{\sqrt{D}}$, la cual luego puede deformarse gracias al parámetro de escala $\alpha \in \mathbb{R}^D$.
- Este método de normalización se ha vuelto usual debido a que se puede computar eficientemente.

Mecanismos de atención

Atención aditiva y multiplicativa

Dada la importancia del mecanismo de atención, encontrar otras maneras definirlo puede mejorar considerablemente la eficiencia y rendimiento de un modelo.

Para un vector query $q_i \in \mathbb{R}^P$ y un vector key $k_j \in \mathbb{R}^P$, se podría definir el score de atención, $s_{ij} \in \mathbb{R}$, de dos formas naturales:

- **Atención aditiva (Bahdanau):** $s_{ij} = W_0 \tanh(W_1 q_i + W_2 k_j)$.
- **Atención multiplicativa:** $s_{ij} = q_i^\top W k_j$.

Scaled dot-product attention (atención estándar) equivale a fijar $W = \frac{1}{\sqrt{P}} I_P$ en la atención multiplicativa.

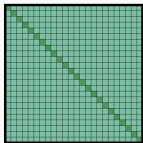
La división por $\sqrt{P}$ se realiza para que $\text{Var}\left(\frac{\langle q_i, k_j \rangle}{\sqrt{P}}\right) = 1$, lo que permite controlar los logits que entrarán a la función softmax.

Problemas del mecanismo de atención estándar

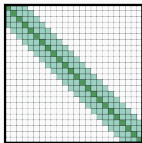
El mecanismo de atención estándar tiene dos problemas principales:

- Su costo computacional es $O(T^2)$, lo que lo hace ineficiente para secuencias largas.
- No es capaz de modelar relaciones de orden superior entre los tokens (solo considera interacciones entre pares de tokens).

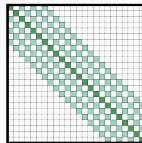
Se han propuesto muchas variantes para estos problemas.



(a) Full n^2 attention



(b) Sliding window attention



(c) Dilated sliding window

Atención cruzada

El mecanismo de self-attention puede ser extendido a un mecanismo de **atención cruzada**, donde una secuencia de embeddings, $(x_1, \dots, x_T) \subset \mathbb{R}^D$ coloca atención sobre otra secuencia de embeddings, $(\tilde{x}_1, \dots, \tilde{x}_{\tilde{T}}) \subset \mathbb{R}^{\tilde{D}}$. Para esto:

- Se calculan los vectores query como $q_i = W_Q x_i \in \mathbb{R}^P$ usando la primera secuencia.
- Se calculan los vectores key y value como $k_j = W_K \tilde{x}_j \in \mathbb{R}^P$ y $v_j = W_V \tilde{x}_j \in \mathbb{R}^P$ usando la segunda secuencia.
- Se aplica el mecanismo de atención usual.

Notar que no es necesario que $T = \tilde{T}$ ni que $D = \tilde{D}$.

Mixture of experts

Luego de un bloque MHSA es usual aplicar un bloque FF, $FF(x_t) = W_2 \phi(W_1 x_t)$, el cual procesa individualmente cada embedding $x_t \in \mathbb{R}^D$ obtenido en el bloque MHSA.

Una extensión es el mecanismo de **mixture of experts (MoE)**:

- Se considera un conjunto de $M \geq 1$ **expertos**, $\{FF_1, \dots, FF_M\}$, cada uno con sus propios parámetros.
- Así, la salida es una combinación convexa de los expertos, donde los ponderadores son determinados por un **enrutador**:

$$\text{MoE}(x_t) = \sum_{m=1}^M \pi_m(x_t) FF_m(x_t) \quad \text{donde} \quad \pi(x_t) = \text{softmax}(W_r x_t)$$

- Para reducir el costo computacional se usa **top-K gating**, donde solo los $K \ll M$ expertos más probables procesan cada x_t .

Dropout y conexiones residuales

Hoy en día, los LLMs modernos se preentrenan durante una sola época, lo que provoca un overfitting mínimo. Por esta razón, los LLMs modernos típicamente no utilizan dropout.

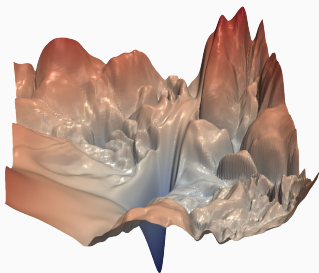
Sin embargo, sí sigue siendo útil en ciertas situaciones:

- Como regularizador en **fine-tuning** usando **adaptadores**.
- Datasets pequeños o con mucho ruido.
- Entrenamiento multiépoca.

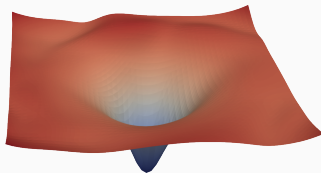
Conexiones residuales

Las **conexiones residuales** $x \mapsto x + \text{Bloque}(x)$ promueven gradientes más estables a lo largo de todas las capas, por lo que ayudan a mitigar el vanishing gradient problem.

Se ha demostrado empíricamente que las conexiones residuales reducen las altas frecuencias en el loss landscape:



(a) without skip connections



(b) with skip connections

En la próxima clase.

- Otros modelos tipo Transformer.

Modelos Generativos Profundos

Clase 7: Modelos autorregresivos y LLMs (parte III)